Встреча 2

# Разработка микросервисных приложений на Golang

YADRO

# Часто встречающиеся ошибки (замечания)

# Часто встречающиеся ошибки: много кода

- сложный код и структура файлов для простой задачи

# Часто встречающиеся ошибки: merge requests

- commit-ы в main без аппрувов и merge requests
- всегда можно исправить - много способов
  - revert
  - branch from last good point
  - force push :)
  - …

# Часто встречающиеся ошибки: storage defaults

- В идеале - через конфиг, но как минимум можно абсолютный путь

# Часто встречающиеся ошибки: user input validation

- Отсутствие валидации имени файла
  - ., /, \n, *

# Часто встречающиеся ошибки: concurrency

- Отсутствие или недостаток обработки одновременных запросов
- Мы это еще будем проходить …
- В типовом решении этого тоже нет

# Часто встречающиеся ошибки: server error

```
if err := http.ListenAndServe(); err ≠ nil {
    slog.Error("http server failed", "error", err)
}

// err всегда non-nil
```

https://pkg.go.dev/net/http#Server.ListenAndServe

# Часто встречающиеся ошибки: server error

```
    // всегда есть ошибка

if err := http.ListenAndServe(); !err ≠ http.ErrServerClosed {
    slog.Error("http server failed", "error", err)
    os.Exit(1)
}
```

# Часто встречающиеся ошибки: cleanenv

```
err := cleanenv.ReadConfig("config.yml", &cfg)
…
err := cleanenv.ReadEnv(&cfg)

// не нужно, если сделать файл обязательным
https://github.com/ilyakaznacheev/cleanenv
```

# Часто встречающиеся ошибки: signal.Notify

```
quit := make(chan os.Signal, 1)
signal.Notify(c, os.Interrupt)
// Block until a signal is received.
←quit

// signal.NotifyContext создает контекст с отменой
https://pkg.go.dev/os/signal#NotifyContext
```

# Часто встречающиеся ошибки: signal.Notify

```
ctx, stop := signal.NotifyContext(
    context.Background(),
    os.Interrupt,
)
defer stop()

go func() {
    <-ctx.Done()
    log.Debug("shutting down server")
    …
```

# Часто встречающиеся ошибки: os.ReadFile, io.ReadAll

```
bytes, err := os.ReadFile(filename) // 10-100G file :)
err := w.Write(bytes)

// лучше сразу читать и записывать
file, err := os.Open(filename)
…
_, err := io.Copy(w, file)
```

# Часто встречающиеся ошибки: mux pre-1.22

```
http.HandleFunc("/files", mySuperMux)

func mySuperMux(w, r) {
    if r.Method == http.MethodGet …
}

// лучше:
http.HandleFunc("GET /files", listFiles)
http.HandleFunc("GET /files/{filename}", getFile)
```

# Часто встречающиеся ошибки: свой (s)log instance?

```
slog.Error / slog.Debug / slog.Info
          - достаточен для микросервиса
```

# Часто встречающиеся ошибки: defer func(x xxx)

```
f, err := os.Open( ... )

defer func(f *os.File) {
    if err := f.Close(); err ≠ nil {
        slog.Error("failed to close file", "error", err)
    }
}(f)
// функции замыканию в данном случае не нужны параметры
```

# Часто встречающиеся ошибки:  нужен ли StatusOK?

- любой write добавляет его автоматом

# Часто встречающиеся ошибки: http.Error() после OK?

```
_, err = io.Copy(w, fileReader) // implicit or explicit http OK

if err != nil {
	slog.Error("failed to copy file", "filepath", filePath, "error", err)

	// поздно пить Коку-Колу, клиент может уже не увидеть следующее
	http.Error(w, "cannot open file", http.StatusInternalServerError)

	// а может и увидеть
}

// https://pkg.go.dev/net/http#ResponseWriter
```

# Часто встречающиеся ошибки: Multipart parsing

```
// 32 MB лимит памяти для парсера, по умолчанию. Необязательно
 err := r.ParseMultipartForm(32 << 20)
 if err ≠ nil {
     http.Error(w, err.Error(), http.StatusInternalServerError)
     return
 }

 // Очистка временных файлов - желательно
 if r.MultipartForm ≠ nil {
 defer r.MultipartForm.RemoveAll()
 }

 file, _, err := r.FormFile("uploadfile") ...
```

# Часто встречающиеся ошибки:  file / folder permissions

- их часто нет
- позволяют больше чем надо

Для директории - 0o775, для файлов - 0o664.

Хороший вариант для  POST:

```
os.OpenFile(filePath, os.O_CREATE|os.O_WRONLY|os.O_EXCL, 0×644)
```

**** Но если вы root… то правила не всегда помогут

# Часто встречающиеся ошибки: Content-Length?

```
// как клиент поймет, что все скачал?

          Transfer-Encoding: chunked

                    vs

          Content-Length: xxxx


// И то и другое хорошо, но второе лучше 😃
```

# Часто встречающиеся ошибки: защита от взлома

```
root, err := os.OpenRoot(fileStorePath)
if err ≠ nil { .. }
defer closeOrLog(root)

fileReader, err := root.Open(filePath)
// https://pkg.go.dev/os#Root - Create, Stat, MkdirAll …
```

# Часто встречающиеся ошибки: проверка ошибок 🙈

Если Close() возвратил ошибку..

4 варианта:

- ошибка клиенту

# Часто встречающиеся ошибки: проверка ошибок 🙈

Если Close() возвратил ошибку..

4 варианта:

- ошибка клиенту - зачем? Он ее поймет? Что с ней делать?

# Часто встречающиеся ошибки: проверка ошибок 🙈

Если Close() возвратил ошибку..

4 варианта:

- ошибка клиенту - зачем? Он ее поймет? Что с ней делать?
- Игнорируем

# Часто встречающиеся ошибки: проверка ошибок 🙈

Если Close() возвратил ошибку..

4 варианта:

- ошибка клиенту - зачем? Он ее поймет? Что с ней делать?
- Игнорируем - в большинстве своем разработчики - оптимисты

# Часто встречающиеся ошибки: проверка ошибок 🙈

Если Close() возвратил ошибку..

4 варианта:

- ошибка клиенту - зачем? Он ее поймет? Что с ней делать?
- Игнорируем - в большинстве своем разработчики - оптимисты
- логируем

# Часто встречающиеся ошибки: проверка ошибок 🙈

Если Close() возвратил ошибку..

4 варианта:

- ошибка клиенту - зачем? Он ее поймет? Что с ней делать?
- Игнорируем - в большинстве своем разработчики - оптимисты
- логируем - знаем, кто что-то не так

# Часто встречающиеся ошибки: проверка ошибок 🙈

Если Close() возвратил ошибку..

4 варианта:

- ошибка клиенту - зачем? Он ее поймет? Что с ней делать?
- Игнорируем - в большинстве своем разработчики - оптимисты
- логируем - знаем, кто что-то не так
- паникуем

# Часто встречающиеся ошибки: проверка ошибок 🙈

Если Close() возвратил ошибку..

4 варианта:

- ошибка клиенту - зачем? Он ее поймет? Что с ней делать?
- Игнорируем - в большинстве своем разработчики - оптимисты
- логируем - знаем, кто что-то не так
- паникуем - не можем сказать, что будет дальше, никаких гарантий, что не навредим

# В помощь “defer f.Close()”

```
// Было:

    defer func() {
        if err ≠ f.Close(); err ≠ nil {
            // ваша стратегия выживания для программы
        }
    }
```

# В помощь “defer f.Close()”

```
// Стало:
import "closer"
    …
    defer closer.CloseOrLog(f, logger)
    defer closer.CloseOrPanic(f)

// Closer package:

    func CloseOrLog(c io.Closer, l *slog.Logger) {
        // close, if err → log
    }
```

# В помощь “defer f.Close()”

```
// Стало:
import "closer"
    …
    defer closer.CloseOrLog(f)
    defer closer.CloseOrPanic(f)

// Closer package:

    func CloseOrLog(c io.Closer) {
        // close, if err → slog
    }
```

# Чем порадовали :)

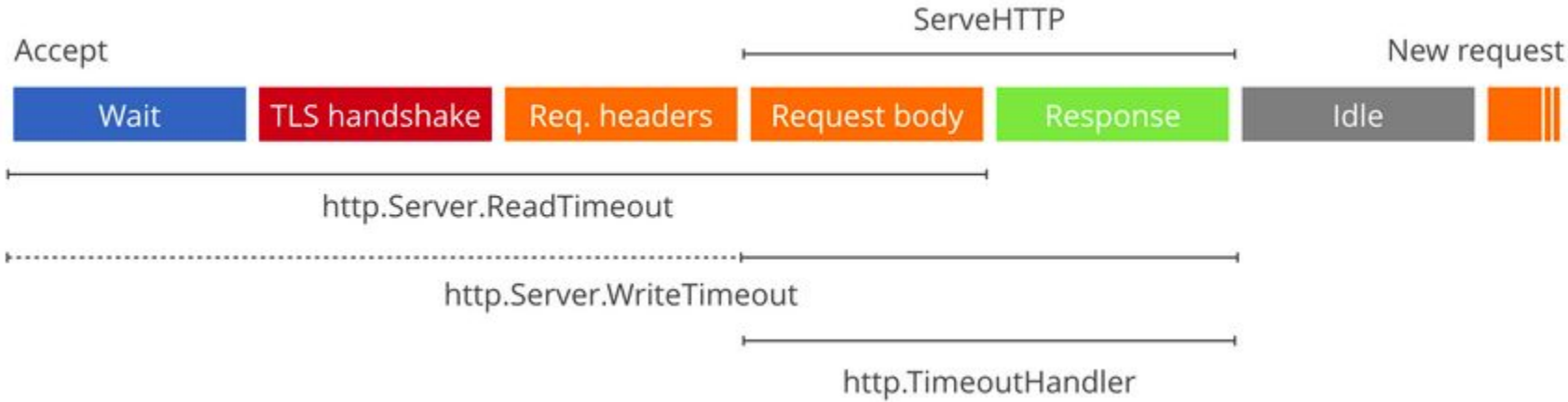
```
func handleDownload(w http.ResponseWriter, r *http.Request){
        http.ServeFile(....)
        …
```

https://pkg.go.dev/net/http#ServeFile

# Останавливать не нужно игнорировать

где поставить запятую…

# HTTP Timeouts




https://kovardin.ru/articles/go/rukovodstvo-po-nethttp-taimautam-v-go/

# Простой сервер с таймаутами

```
func ping(w http.ResponseWriter, r *http.Request) {
       fmt.Fprintln(w, "pong")
}

func main() {
   mux := http.NewServeMux()
   mux.HandleFunc("GET /ping", ping)
```

# Простой сервер с таймаутами

```
server := http.Server{
    Addr:         ":50000",
    ReadTimeout:  5 * time.Second, // include “body” read
    WriteTimeout: 5 * time.Second, // include “body” read
    Handler:      mux,
}
```

# Простой сервер с таймаутами

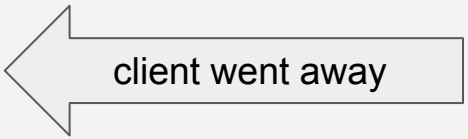
```
err := server.ListenAndServe()
if err != http.ErrServerClosed {
    slog.Error("server failed", "error", err)
    os.Exit(1)
}
```

# Простой сервер с таймаутами

```
func ping(w http.ResponseWriter, r *http.Request) {
    select {
    case <-time.After(10 * time.Second):
        fmt.Fprintln(w, "pong")
    case <-r.Context().Done():
        fmt.Println("context done")
    }
}
```

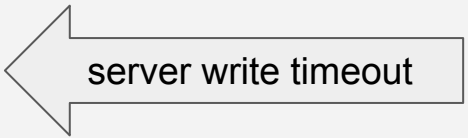
# Простой сервер с таймаутами, Ctrl-C на клиенте

```
func ping(w http.ResponseWriter, r *http.Request) {
    select {
    case ←time.After(10 * time.Second):
        fmt.Fprintln(w, "pong")
    case ←r.Context().Done():
        fmt.Println("context done")
    }
}
```



# Простой сервер с таймаутами, терпеливый клиент

```
func ping(w http.ResponseWriter, r *http.Request) {
    select {
    case ←time.After(10 * time.Second):
        fmt.Fprintln(w, "pong")
    case ←r.Context().Done():
        fmt.Println("context done")
    }
}
```



# Подождем завершения

Пользователь нажимает Ctrl-C:

os.Interrupt

Docker завершает процесс/контейнер:

syscall.SIGTERM

… ждет 10-30 сек

syscall.SIGKILL

# Подождем

```
ctx, stop := signal.NotifyContext(
    context.Background(),
    os.Interrupt, syscall.SIGTERM,
)
defer stop()

server := http.Server{
    BaseContext:  func(l net.Listener) context.Context {
        return ctx
    }, ...
```

# Подождем

Все классно, запросы отменяются, но сервер не останавливается сам

# Подождем

```
go func() {
    <-ctx.Done()
    ctx, cancel := context.WithTimeout(
        context.Background(), 30*time.Second)
    defer cancel()
    if err := server.Shutdown(ctx); err != nil {
        slog.Error("shutdown", "error", err)
    }
}()
// listen and serve here
```

# 3 проблемы

1. base context -> request context Done() one сразу, не ждет завершения handler-ов.
2. ListenAndServe возвращается и main() прибивает горутинку с Shutdown()
3. linting:

   main.go:55:3: exitAfterDefer: os.Exit will exit, and `defer stop()` will not run (gocritic)

   ```
   os.Exit(1)
   ```

# Решение проблемы request context-а

1. Не добавляем base context
2. Проверяем как и прежде request context + добавляем handler timeout

# Решение проблемы ожидания Shutdown

1. wg для Shutdown
2. errgroup

# Решение проблемы ожидания Shutdown

```
group, groupCtx := errgroup.WithContext(ctx)
```

# Решение проблемы ожидания Shutdown

```
group.Go( func() error {
    ←groupCtx.Done()
    shCtx, cancel := context.WithTimeout(
        context.Background(), 60*time.Second)
        defer cancel()
    if err := server.Shutdown(shCtx); err ≠ nil {
        return fmt.Errorf("shutdown failed: %v", err)
    }
    return nil
})
```

# Решение проблемы ожидания Shutdown

```
group.Go(func() error {
    if err := server.ListenAndServe(); err ≠ … {
        return fmt.Errorf("server failed: %v", err)
    }
    return nil
})

return group.Wait()
```

# Решение проблемы defer vs exit

1. exit исключительно в main()
2. func run() error с логикой и defer внутри

```
fun main() {
    if err := run(); err != nil {
        os.Exit(1)
    }
}
```